

TokenTour

Chat to Token

WEBSITE

<https://tokentour.github.io/chat2token/>

GITHUB

<https://github.com/flwdd/TokenTour>

When you chat with an AI—or tell an agent what to do—what actually happens underneath the input box? How do your conversation history, tools, and instructions turn into tokens?

TokenTour is a series of interactive articles, videos, and experiments about large language models and agents. In this first stop, we will look at how one sentence from you becomes something a model can process.

In this article, we will move top-down through Context, Messages, Chat Template, tokens, and KV Cache, using interactive visualizations to make each layer visible.

The first thing to know is that a large language model only does one thing: next-token prediction. In plain language, it plays autocomplete. Given “1 + 1 = ”, it predicts “2”. For now, we do not need to know how the prediction is computed—that is for later stops in the series. Treat the model as a black box: text goes in, the next piece of text comes out.

It has no memory in the human sense, cannot “see” a chat window, and cannot output a whole paragraph all at once. It emits one token, appends that token to everything before it, reads the combined text again, and predicts the next token.

There is no magic hiding under the agent stack. Modern agents can feel nearly omnipotent, but the whole tower is built on top of this autocomplete game. You may already have questions—and if not, the questions below are a good stress test. By the end, they should feel much less mysterious.

🔗 Interactive Playground

This article comes with a browser playground: chat freely with a small agent on the left, while the right side shows synchronized lenses—Chat Template, tokens, and Context × KV Cache (plus the Messages list inside the chat column). Colors and hover highlights are linked across panels, so it is worth keeping open while you read.

🔗 Open Playground →

Messages

🔗 Questions to Keep in Mind

- What is the difference between Context, Messages, and a Prompt?
- How does a model's "memory" work?
- How does the model know which tools exist, and how does it call them?

In a chat UI, you see your messages and the AI's replies. Sometimes you also see the model's reasoning or tool calls. From the model's point of view, how are all of these pieces organized?

The answer is close to what you see: everything is represented as a message list, usually in JSON. Each message has a role, content, and optional extra fields such as reasoning or tool-call data.

There are usually four roles:

- `system`: a system message. There is usually zero or one, set by the developer, used to define the model's persona, behavior preferences, and high-level rules.
- `user`: a user message. It represents what the user sent, and it may also contain extra information injected by the system, such as RAG results.
- `assistant`: an AI message generated by the model, including replies, reasoning, and tool calls.
- `tool`: a tool message generated by the surrounding system, used to pass tool results back to the model.

Content is plain text. The content of system and user messages is usually written by people, and that is what we commonly call the prompt.

A typical message list might look like this. Switch to "Chat" to see how it renders as a conversation:

CHAT

SYSTEM

You are a cheerful cat assistant.

Who are you?

I'm a cheerful cat assistant. Meow.

For simplicity, this example leaves out tools and reasoning. In most chat systems, the system prompt is the first `system` message and is hidden from the user. The model is supposed to follow it, but model outputs are probabilistic, so the model can still drift. For example, even if the system prompt says "do not role-play as a pirate," a later user may pressure the model hard enough that it starts slipping into pirate voice. Similarly, asking "what model are you?" is often not a reliable way to identify the backend. If the system prompt says the assistant is "AcmeBot," it may say it is AcmeBot even when the underlying provider is something else.

At the start, there is only the system message. After you send "Who are you?", the system packages your text as a user message, appends it to the list, and calls the model API. The model generates text, the system packages that text as an assistant message, and appends it too. Every new turn repeats the same loop. So the model does not have "memory" by itself; it simply receives the message list every time. The list records the conversation history, so it behaves as if the model remembered. Everything the model can see is collectively called the context.

For reasoning models, an assistant message may include a reasoning field, commonly separate from the final answer:



assistant message

```
1 {  
2   "role": "assistant",
```

```

3  "content": "I'm a cheerful cat assistant. Meow.",
4  "reasoning_content": "The user asks who I am. I should follow the system instruction and answer briefly."
5  }

```

Tool calls add another layer. Two problems need to be solved: how to tell the model which tools exist, and how to let the model request a tool call. Suppose we have an addition calculator that takes two numbers and returns their sum. How does the model learn the tool exists, and how does it ask to use it? There must be an explicit convention, because the model only deals in text.

Early systems did not have native tool messages or structured tool-call fields. People used carefully designed prompts plus system-side parsing. For example:

early tool-call convention

```

1  [
2  {
3    "role": "system", // Agree with the model on a tool-call format.
4    "content": "When you need to add numbers, output `add(a, b)`, where a and b are numbers."
5  },
6  {
7    "role": "user", // User asks a question.
8    "content": "1 + 1 = ?"
9  },
10 {
11  "role": "assistant", // The model calls a tool in plain text.
12  "content": "add(1, 1)"
13 },
14 {
15  "role": "user", // The system detects the call, runs the tool, and packages the result as a user message.
16  "content": "add(1, 1) = 2"
17 },
18 {
19  "role": "assistant", // The model answers.
20  "content": "2"
21 }
22 ]

```

Here, the [second user message](#) is actually produced by the system. It detected that the model followed the agreed `add(a, b)` format, extracted `1` and `1`, ran the tool, got `2`, and packaged that result back into the message list. This worked, but it was awkward and brittle. Modern APIs make it cleaner, while the underlying idea is the same: define a format the model can learn to emit. When calling the model API, we can include a `tools` field alongside `messages`:

request body (with tools)

```

1  {
2    "messages": [
3      {
4        "role": "user",
5        "content": "1 + 1 = ?"
6      }
7    ],
8    "tools": [
9      {
10     "type": "function",
11     "function": {
12       "name": "add",
13       "description": "Calculate the sum of two numbers",
14       "parameters": {
15         "type": "object",
16         "properties": {
17           "a": { "type": "number" },
18           "b": { "type": "number" }
19         },
20         "required": ["a", "b"]
21       }
22     }
23   ]
24 }

```

```
22     }
23   }
24 ]
25 }
```

Here we declare a tool named `add`. It accepts two numbers and returns their sum. If the model wants to use it, the assistant message can contain a structured tool call:

```
assistant tool_calls

1 {
2   "role": "assistant",
3   "tool_calls": [
4     {
5       "id": "tool_call_id_1",
6       "type": "function",
7       "function": {
8         "name": "add",
9         "arguments": { "a": 1, "b": 1 }
10      }
11    }
12  ]
13 }
```

The system detects that tool call, invokes the tool, receives `2`, and packages the result as a `tool` message. Because multiple tools may be called at once, the result is linked back with `tool_call_id`:

```
tool message

1 {
2   "role": "tool",
3   "content": "2",
4   "tool_call_id": "tool_call_id_1"
5 }
```

Then the system calls the model again with the updated message list. If the model asks for another tool, the loop continues. When it stops calling tools, the model returns the final answer. This is the core of many agents: a loop of reasoning and action, much like the pattern described in Shunyu Yao's [ReAct](#) paper.

Now let's look at a more complete example:

SYSTEM

You are a cheerful cat assistant.

What is today's year × month × day?

▼ Called tool **current_time**

INPUT

{}

OUTPUT

2026-05-30T18:51:09+08:00

▼ Called tool **calculator**

INPUT

```
{
  "expression": "2026 * 5 * 30"
}
```

OUTPUT

303900

The date is 2026-05-30, so year × month × day = $2026 * 5 * 30 = 303900$.

First the model calls `current_time` to get the date, then calls `calculator` with that date, receives `303900`, and finally produces a normal answer without more tool calls.

Whether the product is ChatGPT, Claude Code, or another agent system, a lot of the work comes down to assembling and reassembling this message list. The exact engineering varies, but the shape is surprisingly consistent.

⚡ Try It · Messages Panel

In the playground, open “View JSON” to see the current conversation as a full message list. Edit any message, and the downstream Chat Template, tokens, and Context × KV Cache panels update immediately.

[🔗 Open the playground →](#)

Chat Template

💡 Questions to Keep in Mind

- How do structured Messages become one long plain-text string the model can read?
- What is the difference between a System Prompt and an ordinary User Prompt?
- How does the model distinguish user text from assistant text?
- How is a model's reasoning mode implemented?

The model cannot directly understand a JSON message list. It only understands text. So we need to turn the message list into plain text. This is where the Chat Template enters.

A Chat Template is a formatting template. Apply it to the message list, and it concatenates everything into a single long string in the format that a given model family expects. In the module below, the left side is the message list and the right side is the rendered text after applying different templates. Switch model families to compare them. Hover any role block, message card, or legend chip to link the corresponding parts across both sides; click a legend chip to pin it.

MESSAGES

CHAT

SYSTEM

You are a cheerful cat assistant.

Who are you?

▼ Thinking

The user asks who I am. I should follow the system instruction and answer briefly.

I'm a cheerful cat assistant. Meow.

CHAT TEMPLATE

DEEPSEEK-V3

<|begin_of__sentence|>

You are a cheerful cat assistant.<|User|>Who are you?<|Assistant|> I'm a cheerful cat assistant. Meow.<|end_of__sentence|>

system

user

assistant

The first thing to notice is how different the templates are. DeepSeek V3 is very compact and even omits reasoning content, because that model family does not support reasoning in this format. GPT-OSS is much more elaborate: it injects extra system information on top of the developer's instruction, including identity and channel rules.

Take Qwen3 as an example. We see markers such as `<|im_start|>`. These are special tokens. They separate messages and roles so the model can interpret the stream. The actual input sent to the model looks like:

actual input

```
1 <|im_start|>system
2 You are a cheerful cat assistant.<|im_end|>
3 <|im_start|>user
4 Who are you?<|im_end|>
5 <|im_start|>assistant
6 <think>
```

This tells the model that there is a system message, a user message, and the start of an assistant message. The `<think>` marker says the assistant should reason before answering. The model's generated output is:

model output

```
1 The user asks who I am. I should follow the system instruction and answer briefly.
2 </think>
3
4 I'm a cheerful cat assistant. Meow.<|im_end|>
```

When reasoning is done, the model emits `</think>`, separating reasoning from the final content. When it wants to stop, it emits `<|im_end|>`. Once the system sees `<|im_end|>`, it stops asking for more tokens, appends the model output to the previous text, and parses it back into a message list.

What if we kept asking the model for more tokens? It would probably start producing `<|im_start|>user`. The model only knows how to continue the text. It does not care that the current section is supposed to be an assistant turn; it has learned that assistant messages are often followed by user messages.

How do we control the length of reasoning? In practice, there are soft constraints and hard constraints. A soft constraint uses prompting, such as the injected `Reasoning: medium` in the GPT-OSS system text. It tells the model how much reasoning to do, but the model can still ignore it. A hard constraint uses the template itself. In the Qwen3 example, if we use a “thinking disabled” template, the request sends:

thinking-disabled template

```
1 <|im_start|>system
2 You are a cheerful cat assistant.<|im_end|>
3 <|im_start|>user
4 Who are you?<|im_end|>
5 <|im_start|>assistant
6 <think>
7
8 </think>
```

Now the model starts autocomplete believing the reasoning block has already ended, even if it is empty. It will move directly into the final answer.

Also, even if every assistant message in the message list contains reasoning, a real template may keep only the last reasoning block to save tokens.

Now let’s add tools. The same interactive module shows a multi-turn message list with tool calls on the left and the rendered output for three model families on the right. Pay attention to where the tool schema is inserted, and how tool calls and tool results are represented.

MESSAGES

CHAT

TOOLS

calculator

current_time

SYSTEM

You are a cheerful cat assistant.

What is today's year × month × day?

▼ Called tool **current_time**

INPUT

```
{}
```

OUTPUT

```
2026-05-30T18:51:09+08:00
```

▼ Called tool **calculator**

INPUT

```
{
  "expression": "2026 * 5 * 30"
}
```

OUTPUT

```
303900
```

The date is 2026-05-30, so year × month × day = 2026 * 5 * 30 = 303900.

CHAT TEMPLATE

DEEPSEEK-V3

< | begin__sentence | >

You are a cheerful cat assistant.

Tools

You may call one or more functions to assist with the user query.

```
{
  "type": "function",
  "function": {
    "name": "calculator",
    "description": "Evaluate a basic arithmetic expression. Supports + - * / ( ) and decimal numbers. Returns the numeric result as a string.",
    "parameters": {
      "type": "object",
      "properties": {
        "expression": {
          "type": "string",
          "description": "A pure arithmetic expression, e.g. '12 * (3 + 4) / 2'",
          "required": true
        }
      }
    }
  }
},
{
  "type": "function",
  "function": {
    "name": "current_time",
    "description": "Get the current local date and time in ISO 8601 (with timezone offset).",
    "parameters": {
      "type": "object",
      "properties": {},
      "required": []
    }
  }
}
</tools>
```

For function call returns, you should first print < | tool_calls_begin | >

For each function call, you should return object like:

```
< | tool_call_begin | >function< | tool_sep | ><function_name>
```json
<function_arguments_in_json_format>
```< | tool_call_end | >
```

```
At the end of function call returns, you should print < | tool_calls_end | >< | end_of_sentence | >< | User | >What is today's
year × month × day?< | Assistant | >
< | tool_calls_begin | >< | tool_call_begin | >function< | tool_sep |
>current_time
```json
```

```

"{}"
```< | tool__call__end | > < | tool__calls__end | >< | end_of__sentence | > < | tool__outputs__begin | >< |
tool__output__begin | >2026-05-30T18:51:09+08:00< | tool__output__end | > < | tool__outputs__end | >
| tool__calls__begin | >< | tool__call__begin | >function< | tool__sep | >calculator
```json
{"expression\\":\\"2026 * 5 * 30\\"}"
```< | tool__call__end | > < | tool__calls__end | >< | end_of__sentence | > < | tool__outputs__begin | >< |
tool__output__begin | >303900< | tool__output__end | > < | tool__outputs__end | >The date is 2026-05-30, so year x month x
day = 2026 * 5 * 30 = 303900.< | end_of__sentence | >

```

system

tools schema

user

assistant

tool result

Tool definitions are often placed inside the system prompt. This is not only because the system prompt has strong behavioral priority; it also matters for KV Cache, which we will discuss later.

When the model wants to call a tool, it emits the agreed tool-call format. For example:

```

tool-call format
1 <tool_call>
2 {"name": "calculator", "arguments": {"expression": "2026 * 5 * 30"}}
3 </tool_call>

```

The system can parse that as a request to call `calculator` with `{"expression": "2026 * 5 * 30"}`.

Different templates use wildly different formats: JSON, XML-like tags, function-like calls, and more. The goal is always the same: help the model understand tool use and emit a format the system can parse. The training data often contains many examples of these formats, so the model can learn them.

In general, Chat Template formats are not interchangeable between models. During training, each model family sees a particular format, and at inference time it tends to follow that format. Give it an unfamiliar template and it may produce malformed output.

[Try It · Chat Template Panel](#)

In the playground's Chat Template panel, switch between Qwen3, DeepSeek-V3, and GPT-OSS to compare how the same Messages render into different plain-text inputs. Hover any fragment to highlight its source message and role across panels.

[Open the playground →](#)

Token

[Questions to Keep in Mind](#)

- Why are models bad at counting letters, such as the number of “r” s in strawberry?
- What happens if the user prompt contains a special token?

After the Chat Template step, the model input and output are plain text. But the model still does not process individual characters. It processes tokens.

A token is the model's smallest processing unit. It is often similar to a word or subword. A tokenizer cuts text into tokens, and different tokenizers cut differently. For example, one tokenizer may keep a common English word as one token, while another splits it into several subword pieces. This is true across model families, including DeepSeek V3, Qwen3, and GPT-OSS.

In general, common chunks of text are more likely to become a single token, while rare chunks are more likely to be split into multiple tokens. This reduces the token count statistically over the tokenizer’s training data. Some Unicode characters, including emoji and less common scripts, may even require multiple tokens for one visible character, because tokenization works over UTF-8 bytes rather than abstract characters.

A common modern tokenizer family is BPE: Byte Pair Encoding. Each tokenizer has a vocabulary that maps token IDs to byte sequences. The process begins by encoding the text as UTF-8 bytes, where every byte is its own token. Then the tokenizer repeatedly checks whether adjacent tokens can merge. If a pair can merge, it merges them; if several pairs can merge, it chooses the one with the smallest resulting ID. This continues until no adjacent pair can merge. The final token list is converted to token IDs and sent into the model. The model predicts the next token ID, and the vocabulary converts that ID back into text.

The merge process for `momo` is shown below. Click “Next” to merge step by step, or “Auto play” to watch it move from individual bytes to the final tokens. This uses the real GPT-OSS `o200k` vocabulary. Try a word, a single Chinese character such as `中`, or an emoji; incomplete byte fragments appear as `\xHH`.

BPE Merge Process (GPT-OSS o200k)

momo

m76

om310

o78

m + om → mom

om + o → omo

2 mergeable pairs; choose the lowest resulting ID: om + o → omo (ID 15150)

Step 2 / 3 · 3 tokens

RELEVANT VOCABULARY (6)

m76

o78

om310

mo3690

omo15150

mom94440

STATIC READING NOTES

The PDF freezes a middle merge state so the current sequence, the next candidate merge, and the relevant vocabulary stay visible together. BPE begins at bytes and repeatedly merges the adjacent pair whose merged token has the lowest ID.

0

m

o

m

o

1

m

om

o

ID 316

2

m

omo

ID 15150

The result is two tokens rather than four characters. That is the key static takeaway: the model sees token IDs and byte-derived pieces, not letters directly.

So the tokenizer is not really “splitting” text. It is continuously merging bytes upward.

Different tokenizers also have different vocabulary sizes. For example, DeepSeek V3 has a vocabulary of 128,815 tokens, Qwen3 has 151,669, and GPT-OSS has 201,088. A larger vocabulary can represent more byte sequences as single tokens, so the same text may become fewer tokens—but this is not guaranteed. A tokenizer optimized heavily for one language may perform poorly on another.

This helps explain why models can struggle with counting. The classic question is “How many r’s are in strawberry?” If a tokenizer represents `strawberry` as pieces like `st/raw/berry`, the model sees token IDs, not individual letters. It does not directly see each `r`. Numbers are also token IDs to the model, which is one reason exact arithmetic is hard without tools.

In the Chat Template section, we saw markers such as `<|im_start|>`. These are registered in the vocabulary as special tokens, so the tokenizer treats them as indivisible markers. This is another reason Chat Templates and tokenizers are coupled. If you mix them, a special token may be split into ordinary pieces and the model may fail to understand the structure. If user input contains something that looks like a special token, systems may force it to tokenize as ordinary text to avoid ambiguity.

⚡ Try It · Tokens Panel

In the tokens panel, switch among Qwen3, DeepSeek-V3, GPT-OSS, and other tokenizers. You can inspect each token, its token ID, and the vocabulary size. Hover a token to highlight where it came from in the Chat Template.

🔗 Open the playground →

KV Cache

🔗 Questions to Keep in Mind

- Why should tool definitions and the system prompt be placed near the front?
- Why is the first generated token often much slower than later ones?
- What is a cache hit? Can different users share one?
- How does KV Cache influence agent framework design?

At the beginning, we said the model only does autocomplete: emit one token, append it, reread the whole text, and predict the next token.

Suppose the model has already generated 1000 tokens and now needs token 1001. It would have to read the previous 1000 tokens again. For token 1002, it would read 1001 tokens again, and so on. Without optimization, generating n tokens grows like a triangle, roughly $O(n^2)$. That is terrible.

Fortunately, the model's structure lets us cache part of the previous computation.

If you are curious, we can lift the black-box lid just a little. You can skip this paragraph and still follow the rest. When predicting the next token, the model “looks back” at previous tokens through Attention. More specifically, it computes a pair of vectors for each token: a Key, like an index entry, and a Value, like the content behind that entry. For a new token, the model uses a Query— “what am I looking for?” —to compare against previous Keys, then combines the corresponding Values as evidence for the prediction. We will save the details of Attention for another article. For now, remember this: every token gets a K/V pair.

Because a token can only attend to itself and earlier tokens, its K/V pair does not depend on future tokens. Once computed, it will not change when more tokens are appended. So why recompute it every step? Store it and reuse it. That is KV Cache. The model caches the K and V for every token. When predicting a new token, it computes only the new token's K/V, appends them to the cache, and uses the new Query to read previous K/V from cache. The $O(n^2)$ triangle becomes $O(n)$ work during decoding.

The panel below puts both strategies in the same causal-attention grid. Rows are generation steps; columns are token positions. Only the lower triangle is valid, because a token can see itself and earlier tokens only. Click “Next” to generate one token at a time. On the left, “Without KV Cache” recomputes the whole triangle each step. On the right, “With KV Cache” computes only the new diagonal cell and reads the previous cells from cache. The counters drift apart quickly.

Causal Attention × KV Cache

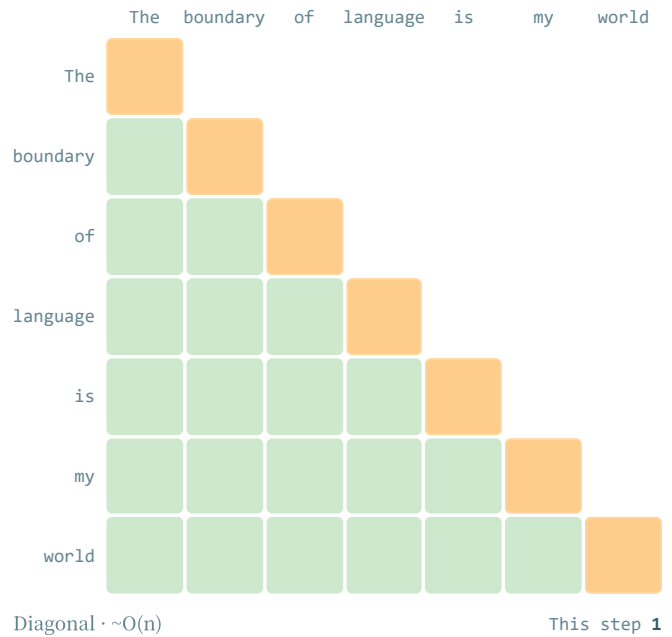
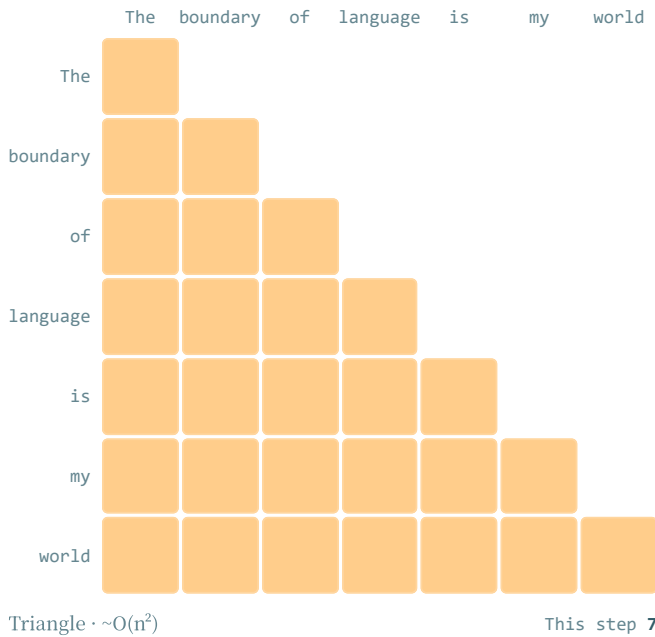
The boundary of language is my world

Without KV Cache

Cumulative recomputes 28

With KV Cache

Cumulative new computes 7



After 7 tokens: without cache, the model computed 28 cells in total (the full triangle, $\sim O(n^2)$); with cache, it computed only 7 cells (the diagonal, $\sim O(n)$). That gap is what KV Cache saves.

■ Computed this step (K/V)
 ■ Read from cache (reuse)
 ■ Not generated yet

One detail: the cache stores K and V, not Q. Each new token creates a fresh Query to read the cache; older Queries are not needed again after their prediction step. That is why it is called “KV Cache,” not “QKV Cache.”

When cache can be reused, computation drops sharply. That is why many model providers price cached input tokens lower than uncached input tokens: a cache hit is much cheaper to serve than recomputing the same prefix.

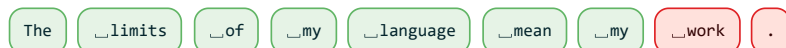
So how does a cache hit happen? The key rule is simple: KV Cache only applies to an identical prefix. Even though a token’s K/V does not depend on what comes after it, it does depend on everything before it. If you change one token in the prefix, every K/V pair after that position may change. The cache can keep only the part before the edit; the rest is invalid and must be recomputed.

The small panel below demonstrates this rule. Type text and it will be tokenized by the real tokenizer. Click “Cache current” to store the token sequence. Then edit the text above. The system searches all cached entries and reuses the longest common prefix: the shared prefix turns green, and everything after the first mismatch turns orange because it must be recomputed. The lower view draws all cached entries as a prefix tree. Shared beginnings are stored once; branches appear only where sequences diverge. Cache a few strings with the same beginning and you will see the footprint become smaller than storing each string independently.

Prefix Cache Hits

Hit rate 78%

The limits of my language mean my work.

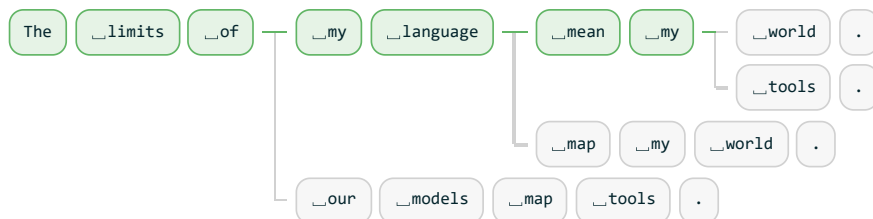


Reused the longest common prefix in the cache tree: the first 7 tokens hit directly; branching starts at token 8, so the remaining 2 need recomputing.

Cache hit (reuse directly) Needs recompute Unused (other cached branches)

CACHE FOOTPRINT · SHARED PREFIXES STORED ONCE

Used 20 / 48 tokens



STATIC CACHE SNAPSHOT

For the PDF export, the script stores prompts that diverge at early, middle, and late tokens before editing the current prompt. The cache tree below should therefore show a shared trunk with branches at several depths.

Try It · Context × KV Cache Panel

The playground's Context × KV Cache panel draws the full context as three colored regions: cached input, uncached input, and output. Edit an early message and the cached-input region shrinks; restore it and the cache hit returns. You can also switch model architectures to compare KV memory footprint per token.

[Open the playground →](#)

Because cache hits depend only on identical prefixes, the same prefix can be reused across requests and even across users. Stable system prompts and tool definitions should go first. Conversation history should be appended after them. If ten thousand users share the same system prompt and tool schema, that public prefix needs to be computed once and can then be reused. This optimization is usually called Prefix Caching.

An efficient agent therefore keeps fixed content—system prompts, tool definitions, policies—pinned near the front, and puts volatile content later. Conversation history should preferably append forward instead of rewriting the middle. In ReAct-style loops, where thought/action cycles make the context grow quickly, stable prefixes save major time and cost. If you put a second-by-second timestamp inside the system prompt, almost every turn starts from a different prefix, which is slow and expensive.

KV Cache is not free. Every token's K/V vectors must be stored in expensive GPU memory, and the cache grows with context length. Its size also depends heavily on model architecture. The industry has invented many tricks around it: GQA and MQA share K/V across attention heads, MLA compresses K/V, and systems may use KV quantization or offloading. We will leave those details for later.

And that completes the journey from a sentence to KV Cache. Your text becomes Messages, the Chat Template compiles those Messages into one long plain-text stream, the tokenizer merges bytes into tokens, and the model predicts one token at a time while caching K/V vectors for reuse. Peel away the impressive agent shell, and underneath is still an autocomplete black box—surrounded by a lot of careful engineering around tokens, context, and cache. Where did the tokens go? Now you know.